

Islamic University of Gaza
Faculty of Engineering
Computer Engineering Department



Database Project

🎓 **Design and Implementation of a Real-World**
Database Management System
📌 **Project Description**
Library Management System

Prepared by :

MLAAK HATEM ALSHAWWA	220210708
AYA SALAH ALASHI	220212933
SONDOS KHAIL	220213320

Dr. Belal Alfara
Eng. Ahmed Alkahlout

Gaza, Palestine

Analysis:

Problem Description

Traditional libraries face problems because they use paper records. This manual work causes mistakes, takes a long time to search for books, and makes it hard to track who borrowed a book or when it is due.

Users

Library Staff: The person who manages the library, updates book records, and tracks borrowing activities.

Borrowers (Students/Guests): The people who search for books, borrow them, and return them.

Functional Requirements

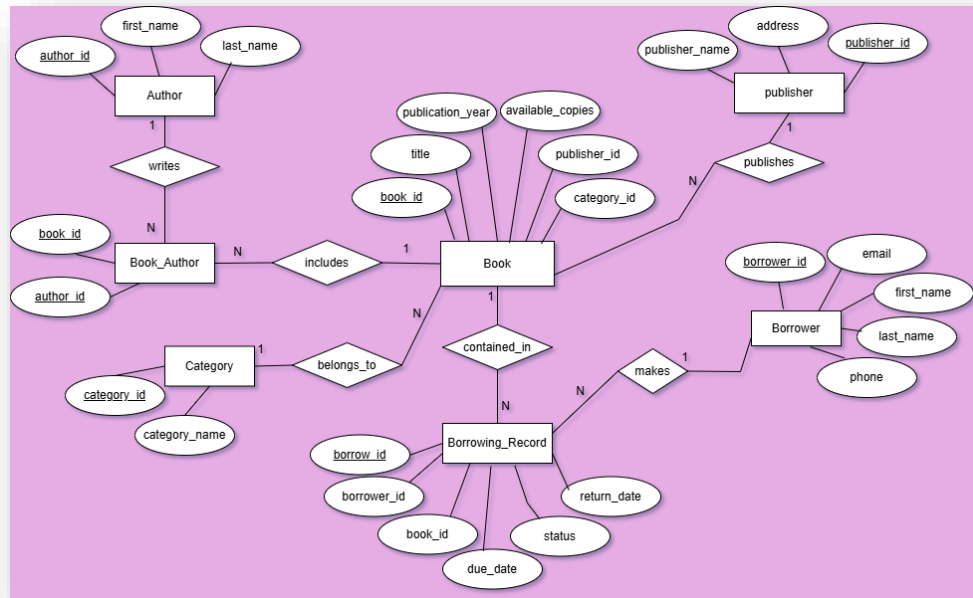
- ➔ Store book information in the database.
- ➔ Store borrower information.
- ➔ Record borrowing transactions using the borrowing_record table.
- ➔ Maintain relationships between books, borrowers, authors, publishers, and categories.
- ➔ Retrieve information using SQL queries.

Generate reports and summaries using SQL aggregate functions.

Objectives

- ➔ **Save Time:** To make searching and recording much faster.
- ➔ **Reduce Mistakes:** To eliminate errors caused by manual paperwork.
- ➔ **Easy Tracking:** To easily track available and overdue books.

ER Diagram:



Relational Schema (Logical Design)

1. borrower

borrower(borrower_id, first_name, last_name, email, phone)

PK: borrower_id

2. book

book(book_id, title, publication_year, available_copies,
category_id, publisher_id)

PK: book_id

FK: category_id references category(category_id)

FK: publisher_id references publisher(publisher_id)

3. author

author(author_id, first_name, last_name)

PK: author_id

4. category

category(category_id, category_name)

PK: category_id

5. publisher

publisher(publisher_id, publisher_name, address)

PK: publisher_id

6. borrowing_record

borrowing_record(borrow_id, borrower_id, book_id, due_date,
return_date, status)

PK: borrow_id

FK: borrower_id references borrower(borrower_id)

FK: book_id references book(book_id)

7. book_author

book_author(book_id, author_id)

PK: (book_id, author_id)

FK: book_id references book(book_id)

FK: author_id references author(author_id)

Relationships

Borrower **1** ---- **M** Borrowing_Record

Book **1** ---- **M** Borrowing_Record

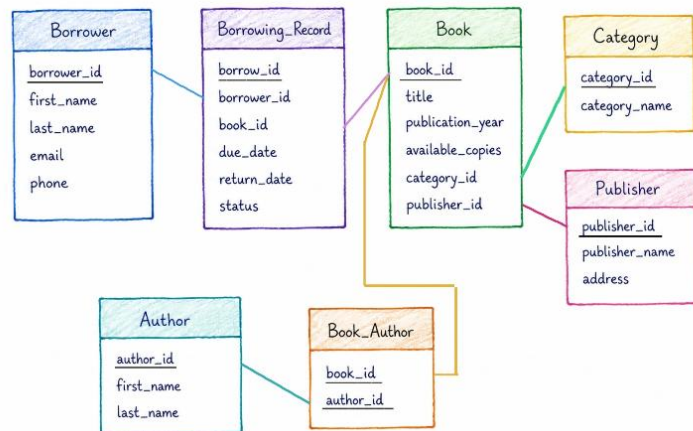
Publisher **1** ---- **M** Book

Category **1** ---- **M** Book

Book **M** ---- **N** Author

.... Book **1** ---- **M** Book_Author

..... Author **1** ---- **M** Book_Author



Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity by dividing data into related tables.

Why Do We Use Normalization?

- Reduce duplicated data.
- Improve database organization.
- Prevent insertion, deletion, and update problems.
- Make data management easier.

UNF (Unnormalized Form)

At the beginning, all library data was stored in one large table containing borrower, book, author, category, and publisher information.

Problems:

- Data redundancy
- Difficulty updating data
- Insertion and deletion anomalies

Ex:

borrower_id	borrower_name	email	phone	book_id	book_title	publication_year	author_name	category_name	publisher_name
1	Aya Salah Alashi	aya@gmail.com	0599112233	1	Data Structures	2022	Omar Nasser	Programming	Pearson
2	Malak Hatem Alshawa	malak@gmail.com	0599445566	2	Artificial Intelligence	2023	Lina Ahmad	AI	McGraw Hill

🚦 1NF(First Normal Form)

- Each attribute contains atomic values.
- No repeating groups exist.
- Each row is unique.

The data was separated into different tables such as:

Borrower , Book , Author , CategoryPublisher

Ex:

Borrower Table

borrower_id	first_name	last_name	email	phone
1	Aya	Alashi	aya@gmail.com	0599112233
2	Malak	Alshawa	malak@gmail.com	0599445566

Book Table

book_id	title	publication_year	available_copies	category_id	publisher_id
1	Data Structures	2022	4	1	1
2	Artificial Intelligence	2023	6	2	2

Author Table

author_id	first_name	last_name
1	Omar	Nasser
2	Lina	Ahmad

🚦 2NF (Second Normal Form)

the table must first be in 1NF, and all attributes should depend on the whole primary key.

In our project, the book_author table uses a composite primary key:

(book_id, author_id)

This helps reduce data redundancy and organize the relationship between books and authors.

Ex:

Book_Author Table

book_id	author_id
1	1
2	2
2	2

🌈 3NF (Third Normal Form)

the table must first be in 2NF, and transitive dependency should be removed.

In our project, we created separate tables such as category and publisher instead of storing their data repeatedly inside the book table.

Category Table

category_id	category_name
1	Programming
2	AI

Publisher Table

publisher_id	publisher_name	address
1	Pearson	London
2	McGraw Hill	New York

Book Table

book_id	title	publication_year	available_copies	category_id	publisher_id
1	Data Structures	2022	4	1	1
2	Artificial Intelligence	2023	6	2	2

Final Result

- After applying normalization:
- Data redundancy was minimized.
- Database consistency improved.
- Data became easier to manage.
- The database structure became more efficient and organized.

SQL Implementation (Physical Design)

- DDL (Create Tables):

```
library_system/postgres@PostgreSQL 18* X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table borrower(
2 borrower_id int primary key,
3 first_name varchar(50) not null,
4 last_name varchar(50) not null,
5 email varchar(100),
6 phone varchar(20)
7 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 374 msec.
```

```
library_system/postgres@PostgreSQL 18* X library_system/po... X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table book(
2 book_id int primary key,
3 title varchar(150) not null,
4 publication_year int,
5 available_copies int,
6 category_id int,
7 publisher_id int,
8
9 foreign key (category_id) references category(category_id),
10 foreign key (publisher_id) references publisher(publisher_id)
11 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 74 msec.
```

```
library_system/postgres@PostgreSQL 18* X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table pulisher(
2 publisher_id int primary key,
3 publisher_name varchar(100) not null,
4 address varchar(200)
5 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 94 msec.
```

```
library_system/postgres@PostgreSQL 18* X library_system/po... X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table borrowing_record(
2 borrow_id int primary key,
3 borrower_id int,
4 book_id int,
5 due_date date,
6 return_date date,
7 status varchar(30),
8
9 foreign key (borrower_id) references borrower(borrower_id),
10 foreign key (book_id) references book(book_id)
11 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 105 msec.
```

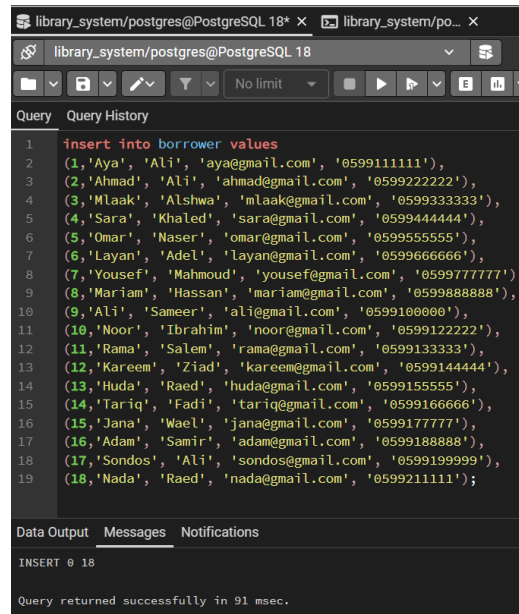
```
library_system/postgres@PostgreSQL 18* X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table author(
2 author_id int primary key,
3 first_name varchar(50) not null,
4 last_name varchar(50) not null
5 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 130 msec.
```

```
library_system/postgres@PostgreSQL 18* X library_system/po... X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table book_author(
2 book_id int,
3 author_id int,
4 primary key (book_id,author_id),
5 foreign key (book_id) references book(book_id),
6 foreign key (author_id) references author(author_id)
7 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 73 msec.
```

```
library_system/postgres@PostgreSQL 18* X
library_system/postgres@PostgreSQL 18
Query Query History
1 create table category(
2 category_id int primary key,
3 category_name varchar(100) not null
4 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 112 msec.
```


- **DML (Insert Into):**

- **To borrower**



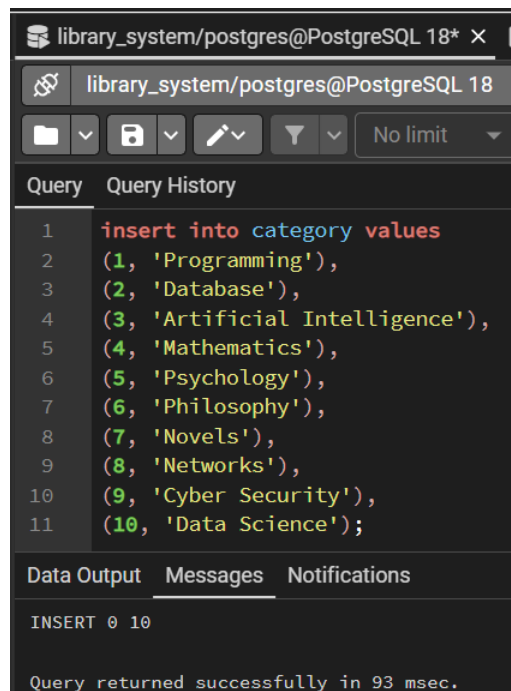
The screenshot shows a PostgreSQL query editor window titled 'library_system/postgres@PostgreSQL 18*'. The query is an INSERT statement into the 'borrower' table, listing 18 rows of data. The data includes names, email addresses, and phone numbers. The 'Data Output' tab shows 'INSERT 0 18' and a message 'Query returned successfully in 91 msec.'

```
1 insert into borrower values
2 (1,'Aya', 'Ali', 'aya@gmail.com', '0599111111'),
3 (2,'Ahmad', 'Ali', 'ahmad@gmail.com', '0599222222'),
4 (3,'Mlaak', 'Alshwa', 'mlaak@gmail.com', '0599333333'),
5 (4,'Sara', 'Khaled', 'sara@gmail.com', '0599444444'),
6 (5,'Omar', 'Naser', 'omar@gmail.com', '0599555555'),
7 (6,'Layan', 'Adel', 'layan@gmail.com', '0599666666'),
8 (7,'Yousef', 'Mahmoud', 'yousef@gmail.com', '0599777777'),
9 (8,'Mariam', 'Hassan', 'mariam@gmail.com', '0599888888'),
10 (9,'Ali', 'Sameer', 'ali@gmail.com', '0599100000'),
11 (10,'Noor', 'Ibrahim', 'noor@gmail.com', '0599122222'),
12 (11,'Rama', 'Salem', 'rama@gmail.com', '0599133333'),
13 (12,'Kareem', 'Ziad', 'kareem@gmail.com', '0599144444'),
14 (13,'Huda', 'Raed', 'huda@gmail.com', '0599155555'),
15 (14,'Tariq', 'Fadi', 'tariq@gmail.com', '0599166666'),
16 (15,'Jana', 'Wael', 'jana@gmail.com', '0599177777'),
17 (16,'Adam', 'Samir', 'adam@gmail.com', '0599188888'),
18 (17,'Sondos', 'Ali', 'sondos@gmail.com', '0599199999'),
19 (18,'Nada', 'Raed', 'nada@gmail.com', '0599211111');
```

INSERT 0 18

Query returned successfully in 91 msec.

- **To category**



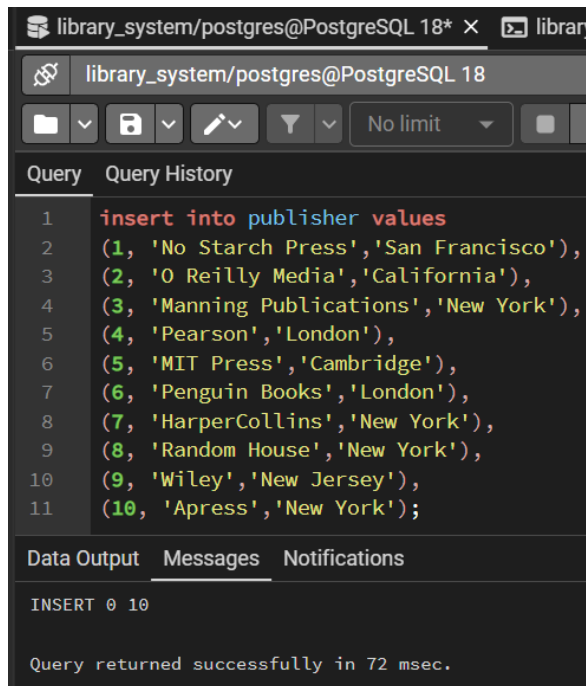
The screenshot shows a PostgreSQL query editor window titled 'library_system/postgres@PostgreSQL 18*'. The query is an INSERT statement into the 'category' table, listing 10 rows of data. The data includes category names. The 'Data Output' tab shows 'INSERT 0 10' and a message 'Query returned successfully in 93 msec.'

```
1 insert into category values
2 (1, 'Programming'),
3 (2, 'Database'),
4 (3, 'Artificial Intelligence'),
5 (4, 'Mathematics'),
6 (5, 'Psychology'),
7 (6, 'Philosophy'),
8 (7, 'Novels'),
9 (8, 'Networks'),
10 (9, 'Cyber Security'),
11 (10, 'Data Science');
```

INSERT 0 10

Query returned successfully in 93 msec.

- **To publisher:**

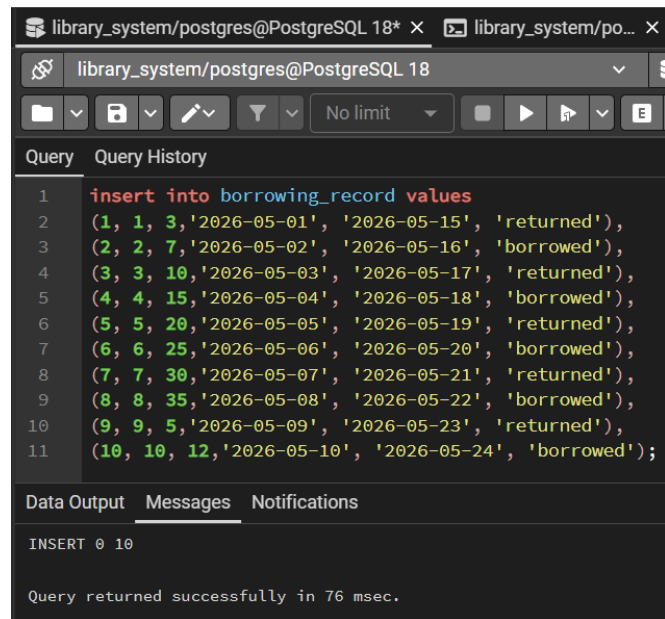


The screenshot shows a PostgreSQL query editor window with the title 'library_system/postgres@PostgreSQL 18*'. The query is an insert statement for the 'publisher' table. The query is as follows:

```
1 insert into publisher values
2 (1, 'No Starch Press','San Francisco'),
3 (2, 'O Reilly Media','California'),
4 (3, 'Manning Publications','New York'),
5 (4, 'Pearson','London'),
6 (5, 'MIT Press','Cambridge'),
7 (6, 'Penguin Books','London'),
8 (7, 'HarperCollins','New York'),
9 (8, 'Random House','New York'),
10 (9, 'Wiley','New Jersey'),
11 (10, 'Apress','New York');
```

The query is executed successfully, and the output shows 'INSERT 0 10' and 'Query returned successfully in 72 msec.'

- **To borrowing_record**



The screenshot shows a PostgreSQL query editor window with the title 'library_system/postgres@PostgreSQL 18*'. The query is an insert statement for the 'borrowing_record' table. The query is as follows:

```
1 insert into borrowing_record values
2 (1, 1, 3, '2026-05-01', '2026-05-15', 'returned'),
3 (2, 2, 7, '2026-05-02', '2026-05-16', 'borrowed'),
4 (3, 3, 10, '2026-05-03', '2026-05-17', 'returned'),
5 (4, 4, 15, '2026-05-04', '2026-05-18', 'borrowed'),
6 (5, 5, 20, '2026-05-05', '2026-05-19', 'returned'),
7 (6, 6, 25, '2026-05-06', '2026-05-20', 'borrowed'),
8 (7, 7, 30, '2026-05-07', '2026-05-21', 'returned'),
9 (8, 8, 35, '2026-05-08', '2026-05-22', 'borrowed'),
10 (9, 9, 5, '2026-05-09', '2026-05-23', 'returned'),
11 (10, 10, 12, '2026-05-10', '2026-05-24', 'borrowed');
```

The query is executed successfully, and the output shows 'INSERT 0 10' and 'Query returned successfully in 76 msec.'

- **To author,book,book_author:**

```

1 insert into author values
2 (1,'Eric', 'Matthes'),
3 (2,'Al', 'Sweigart'),
4 (3,'Dan', 'Vanderkam'),
5 (4,'Luciano', 'Ramalho'),
6 (5,'Jon', 'Gjengset');
7
8 insert into book values
9 (1,'Python Crash Course', 2023, 8, 1, 1),
10 (2,'Automate the Boring Stuff with Python', 2020, 6, 1, 1),
11 (3,'Effective TypeScript', 2024, 5, 1, 2),
12 (4,'Fluent Python', 2022, 4, 1, 2),
13 (5,'Rust for Rustaceans', 2021, 3, 1, 1);
14
15 insert into book_author values
16 (1,1),
17 (2,2),
18 (3,3),
19 (4,4),
20 (5,5);

```

Query returned successfully in 72 msec.

The same insertion method was used to add the remaining 33 books along with their authors and relationships in the database tables.

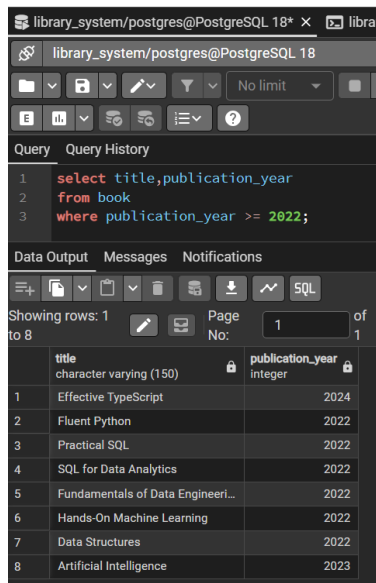
In addition, some values were modified and updated later to improve the consistency and organization of the database.

- **Queries:**

book_id [PK] integer	title character varying (150)	publication_year integer	available_copies integer	category_id integer	publisher_id integer
1	Effective TypeScript	2024	5	1	2
2	Fluent Python	2022	4	1	2
3	Rust for Rustaceans	2021	3	1	1
4	SQL for Data Analysis	2021	5	2	2
5	Practical SQL	2022	7	2	1
6	Database Internals	2020	4	2	2
7	SQL for Data Analytics	2022	6	2	9
8	Fundamentals of Data Engineering	2022	5	2	2
9	Artificial Intelligence: A Modern App...	2020	4	3	4
10	Deep Learning with Python	2021	6	3	3
11	Hands-on Machine Learning	2022	5	3	2
12	Machine Learning Engineering	2020	3	3	10
13	The Algorithm Problem	2020	4	3	6
14	Linear Algebra and Its Applications	2021	5	4	4
15	Infinite Powers	2020	4	4	6
16	Do Dice Play God?	2020	3	4	7
17	How Not to Be Wrong	2021	6	4	8
18	Measurement	2020	5	4	6
19	Thinking, Fast and Slow	2021	5	5	6
20	Atomic Habits	2020	8	5	8
21	Thinking, Fast and Slow	2021	5	5	6
22	Emotional Intelligence	2020	4	5	7
23	The Psychology of Money	2021	6	5	9
24	Surrounded by Idiots	2020	5	5	6
25	Meditations	2020	6	6	6
26	The Republic	2021	4	6	7
27	Beyond Good and Evil	2020	3	6	8
28	Sophies World	2021	5	6	6
29	A Little History of Philosophy	2020	4	6	9
30	The Alchemist	2020	7	7	6
31	The Silent Patient	2021	5	7	8
32	It Ends With Us	2020	6	7	7
33	The Midnight Library	2021	4	7	6
34	The Seven Husbands of Evelyn Hugo	2020	5	7	8
35	Data Structures	2022	4	1	1
36	Artificial Intelligence	2023	6	2	2
37	Data Science from Scratch	2021	5	10	2
38	Computer Networking: A Top-Down Ap...	2021	4	8	4
39	Hacking: The Art of Exploitation	2021	3	9	1

This query was used to display all books stored in the database along with their information such as title, publication year, available copies, category ID, and publisher ID.

The result confirms that the book records were inserted successfully into the system tables.



The screenshot shows a PostgreSQL query editor with the following query:

```

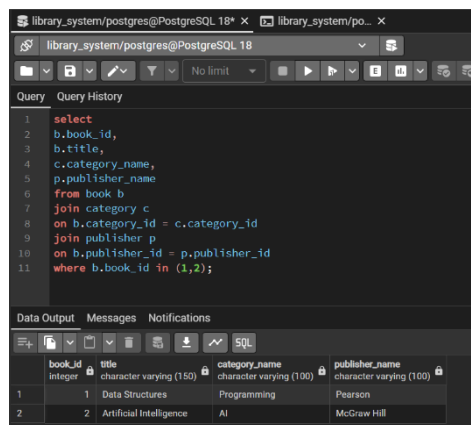
1 select title, publication_year
2 from book
3 where publication_year >= 2022;

```

The results table shows 8 rows of data:

	title	publication_year
1	Effective TypeScript	2024
2	Fluent Python	2022
3	Practical SQL	2022
4	SQL for Data Analytics	2022
5	Fundamentals of Data Engineeri...	2022
6	Hands-On Machine Learning	2022
7	Data Structures	2022
8	Artificial Intelligence	2023

This query displays the titles and publication years of books published in or after 2022. It was used to filter and retrieve recently published books from the database.



The screenshot shows a PostgreSQL query editor with the following query:

```

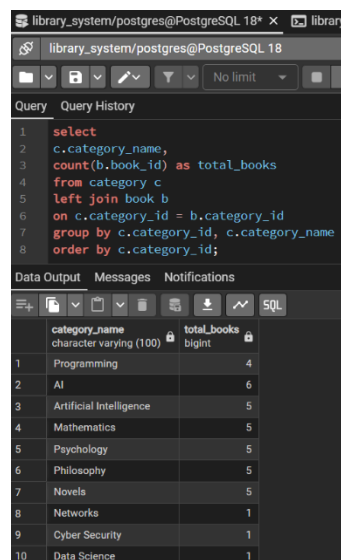
1 select
2 b.book_id,
3 b.title,
4 c.category_name,
5 p.publisher_name
6 from book b
7 join category c
8 on b.category_id = c.category_id
9 join publisher p
10 on b.publisher_id = p.publisher_id
11 where b.book_id in (1,2);

```

The results table shows 2 rows of data:

	book_id	title	category_name	publisher_name
1	1	Data Structures	Programming	Pearson
2	2	Artificial Intelligence	AI	McGraw Hill

This query uses JOIN operations to display book information together with the related category and publisher names. The result shows how the relationships between the tables were successfully implemented in the database.



The screenshot shows a PostgreSQL query editor with the following query:

```

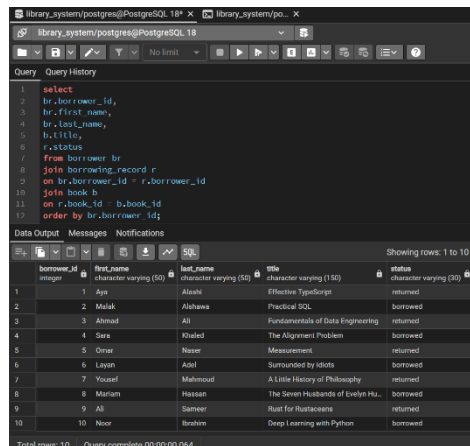
1 select
2 c.category_name,
3 count(b.book_id) as total_books
4 from category c
5 left join book b
6 on c.category_id = b.category_id
7 group by c.category_id, c.category_name
8 order by c.category_id;

```

The results table shows 10 rows of data:

	category_name	total_books
1	Programming	4
2	AI	6
3	Artificial Intelligence	5
4	Mathematics	5
5	Psychology	5
6	Philosophy	5
7	Novels	5
8	Networks	1
9	Cyber Security	1
10	Data Science	1

This query counts the number of books available in each category using GROUP BY and aggregate functions. It was used to organize and summarize the data stored in the library system.



The screenshot shows a PostgreSQL query editor with a query window and a results window. The query window contains a SQL query that joins the borrower, borrowing_record, and book tables. The results window displays the output of the query, showing 10 rows of data.

```

1 select
2   br.borrower_id,
3   br.first_name,
4   br.last_name,
5   b.title,
6   r.status
7 from borrower br
8 join borrowing_record r
9   on r.borrower_id = br.borrower_id
10 join book b
11   on r.book_id = b.book_id
12 order by br.borrower_id;

```

borrower_id	first_name	last_name	title	status
1	Aya	Alshabi	Effective TypeScript	returned
2	Malak	Alshams	Practical SQL	borrowed
3	Ahmad	Ali	Fundamentals of Data Engineering	returned
4	Sara	Khaled	The Alignment Problem	borrowed
5	Omar	Naser	Measurement	returned
6	Layth	Adel	Surrounded by Idiots	borrowed
7	Yousef	Mahmoud	A Little History of Philosophy	returned
8	Mariam	Hassan	The Seven Habits of Evelyn Ho...	borrowed
9	Ali	Samer	Read for Rustaceans	returned
10	Nour	Abdine	Deep Learning with Python	borrowed

This query uses JOIN operations to display borrower information together with the borrowed books and borrowing status.

The result confirms that the relationships between borrower, book, and borrowing_record tables are working correctly in the database system.

Conclusion

In this project, we designed and implemented a complete Library Management Database System using PostgreSQL. The system was built using multiple related tables including books, authors, borrowers, categories, publishers, and borrowing records. We applied different SQL operations such as DDL, DML, JOIN queries, filtering, and aggregate functions to manage and retrieve the data efficiently.

Throughout the project, we focused on organizing the database structure correctly and creating clear relationships between the tables to ensure consistency and accuracy of the stored data. In addition, the project helped us improve our practical understanding of database concepts, relational schema design, normalization, and SQL query implementation in a real and organized system.